

◆ Angular Forms Overview (Angular 20+)

Angular provides two distinct paradigms for handling forms:

1. **Template-Driven Forms** → Declarative, HTML-centric
2. **Reactive Forms** → Programmatic, TypeScript-centric

Both rely on the `@angular/forms` package but differ in architecture, scalability, and control.

◆ 1. Template-Driven Forms (Overview)

✓ Concept

Template-driven forms are built using **Angular directives in HTML**. Angular internally creates and manages the form model.

- Uses `ngModel`
 - Uses directives like `ngForm`, `ngModelGroup`
 - Suitable for **simple forms**
-

🧠 Flow

HTML Template → Angular Directives → Form Model (created internally)

📦 Setup

Import in your module:

```
import { FormsModule } from '@angular/forms';
```

```
@NgModule({  
  imports: [FormsModule]  
})  
  
export class AppModule {}
```

Practical Example

◆ HTML

```
<form #userForm="ngForm" (ngSubmit)="onSubmit(userForm)">
```

```
<label>Name:</label>
```

```
<input type="text" name="name" ngModel required>
```

```
<label>Email:</label>
```

```
<input type="email" name="email" ngModel required>
```

```
<button type="submit">Submit</button>
```

```
</form>
```

◆ Component (TS)

```
onSubmit(form: any) {  
  console.log(form.value);  
}
```

Key Observations

- #userForm="ngForm" → creates form reference
 - ngModel → binds input fields
 - Validation is declared in HTML (required)
-

When to Use

- Small forms

- Less validation logic
 - Faster development
-

◆ 2. Reactive Forms (Overview)

✅ Concept

Reactive forms are **model-driven**. You explicitly define the form structure in **TypeScript**.

- Uses FormGroup, FormControl, FormBuilder
 - Highly scalable and testable
 - Preferred for **enterprise apps**
-

🧠 Flow

Component (TS Form Model) → Bind to Template → Sync UI

📦 Setup

```
import { ReactiveFormsModule } from '@angular/forms';
```

```
@NgModule({  
  imports: [ReactiveFormsModule]  
})  
  
export class AppModule {}
```

🔧 Practical Example

◆ Component (TS)

```
import { FormGroup, FormControl, Validators } from '@angular/forms';
```

```
export class AppComponent {  
  
  userForm = new FormGroup({  
    name: new FormControl("", Validators.required),  
    email: new FormControl("", [Validators.required, Validators.email])  
  });  
  
  onSubmit() {  
    console.log(this.userForm.value);  
  }  
}
```

◆ HTML

```
<form [formGroup]="userForm" (ngSubmit)="onSubmit()">  
  <label>Name:</label>  
  <input type="text" formControlName="name">  
  <label>Email:</label>  
  <input type="email" formControlName="email">  
  <button type="submit">Submit</button>  
</form>
```

Key Observations

- formGroup → binds form model
 - formControlName → binds controls
 - Validation defined in **TypeScript**
-

✓ When to Use

- Complex forms
 - Dynamic forms
 - Custom validations
 - Large-scale applications
-

● Key Differences (Template vs Reactive)

Feature	Template-Driven	Reactive Forms
Approach	HTML-driven	TypeScript-driven
Form Model	Created by Angular	Explicitly created
Data Flow	Two-way binding	One-way reactive flow
Validation	In template	In component
Scalability	Low	High
Testing	Difficult	Easy
Debugging	Harder	Easier
Learning Curve	Easy	Moderate

🏛 Quick Summary

- **Template-Driven Forms** → Simpler, less code, good for beginners
 - **Reactive Forms** → Structured, powerful, best for real-world apps
-

◆ 1. Template-Driven Form Validation

✓ Concept

Validation is declared directly in the **HTML template** using:

- HTML5 attributes (required, minlength, etc.)
 - Angular directives (ngModel, ngModelGroup)
-

🔧 Practical Example (Login Form)

◆ HTML

```
<form #loginForm="ngForm" (ngSubmit)="onSubmit(loginForm)">
```

```
<label>Email:</label>
```

```
<input
```

```
  type="email"
```

```
  name="email"
```

```
  ngModel
```

```
  required
```

```
  email
```

```
  #email="ngModel">
```

```
<div *ngIf="email.invalid && email.touched">
```

```
  <small *ngIf="email.errors?.['required']">Email is required</small>
```

```
  <small *ngIf="email.errors?.['email']">Invalid email format</small>
```

```
</div>
```

```
<label>Password:</label>
```

```
<input
```

```
type="password"
name="password"
ngModel
required
minlength="6"
#password="ngModel">
```

```
<div *ngIf="password.invalid && password.touched">
  <small *ngIf="password.errors?.['required']">Password is required</small>
  <small *ngIf="password.errors?.['minlength']">
    Minimum 6 characters required
  </small>
</div>

<button [disabled]="loginForm.invalid">Login</button>

</form>
```

◆ Component

```
onSubmit(form: any) {
  console.log(form.value);
}
```

Key Concepts

- #email="ngModel" → exposes control state
- email.invalid, email.touched

- errors object contains validation failures
-

Limitations

- Logic mixed in HTML
 - Hard to scale for complex rules
 - Not ideal for dynamic validations
-

2. Reactive Form Validation

Concept

Validation is defined in **TypeScript**, giving full control.

Practical Example (Login Form)

Component (TS)

```
import { FormGroup, FormControl, Validators } from '@angular/forms';
```

```
export class AppComponent {
```

```
  loginForm = new FormGroup({  
    email: new FormControl("", [  
      Validators.required,  
      Validators.email  
    ]),  
    password: new FormControl("", [  
      Validators.required,  
      Validators.minLength(6)  
    ])  
  });
```



```
});
```

```
onSubmit() {  
  console.log(this.loginForm.value);  
}  
}
```

◆ HTML

```
<form [formGroup]="loginForm" (ngSubmit)="onSubmit()">
```

```
<label>Email:</label>
```

```
<input type="email" formControlName="email">
```

```
<div *ngIf="loginForm.get('email')?.invalid && loginForm.get('email')?.touched">
```

```
<small *ngIf="loginForm.get('email')?.errors?.['required']">
```

```
  Email is required
```

```
</small>
```

```
<small *ngIf="loginForm.get('email')?.errors?.['email']">
```

```
  Invalid email
```

```
</small>
```

```
</div>
```

```
<label>Password:</label>
```

```
<input type="password" formControlName="password">
```

```
<div *ngIf="loginForm.get('password')?.invalid && loginForm.get('password')?.touched">
```

```
<small *ngIf="loginForm.get('password')?.errors?.['required']">
```

Password is required

```
</small>
```

```
<small *ngIf="loginForm.get('password')?.errors?.['minlength']">
```

Min 6 characters

```
</small>
```

```
</div>
```

```
<button [disabled]="loginForm.invalid">Login</button>
```

```
</form>
```
